

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 11 · LECTURE NOTES

Network Flow and Matching

One algorithm, an implausible number of applications.

MODULE OUTCOMES

- State the max-flow min-cut theorem and explain why it holds.
- Implement Ford–Fulkerson and Edmonds–Karp.
- Explain residual graphs and why reverse edges are necessary.
- Model bipartite matching, project selection, and segmentation as flow.
- Recognise when a problem is secretly a flow problem.

1 Flow networks

A flow network is a directed graph with a non-negative capacity on each edge, a source s , and a sink t . A **flow** assigns a value to each edge subject to two constraints:

- **Capacity:** $0 \leq f(u,v) \leq c(u,v)$ on every edge.
- **Conservation:** at every vertex other than s and t , total flow in equals total flow out.

The **value** of the flow is the net flow out of s , and the problem is to maximise it. The physical metaphor — water through pipes — is accurate and worth keeping, because the intuitions it supplies are correct.

1.1 Cuts

An **s – t cut** partitions the vertices into (S, T) with $s \in S$ and $t \in T$. Its capacity is the sum of capacities of edges going from S to T (edges in the reverse direction do not count).

Every unit of flow from s to t must cross every such cut, so the value of any flow is at most the capacity of any cut. That direction is immediate. The content of the theorem is that the bound is tight.

MAX-FLOW MIN-CUT THEOREM

The maximum value of an s – t flow equals the minimum capacity of an s – t cut.

The proof is constructive: when the augmenting-path algorithm terminates, the set of vertices still reachable from s in the residual graph forms one side of a minimum cut, and its capacity equals the flow value.

This means you do not merely know the two numbers are equal — the algorithm hands you the actual cut. Many applications want the cut and treat the flow as a means to it.

2 Ford–Fulkerson and the residual graph

The algorithm is almost trivially simple: while there is a path from s to t with spare capacity, push flow along it. The difficulty is that a naive version of this is greedy and greedy fails — an early routing choice can block the optimum, exactly as in Module 09. The residual graph is the repair.

2.1 Residual capacities

For each edge (u,v) carrying flow f out of capacity c , the residual graph contains:

- a **forward** edge (u,v) with residual capacity $c - f$ — how much more you could send;
- a **reverse** edge (v,u) with residual capacity f — how much you could take back.

THE REVERSE EDGE IS THE WHOLE IDEA

Sending flow along a reverse edge means *cancelling* flow you previously committed. It lets a later augmenting path reroute earlier decisions as a side effect of pushing new flow.

Without reverse edges the algorithm is greedy and can stall below the optimum. With them, the theorem guarantees that if no augmenting path exists the flow is maximum — there is no longer any way to be stuck in a locally-committed state.

If you remember one thing from this module, remember that flow is *greedy plus the ability to undo*, and that the undo is implemented as an edge in a derived graph.

2.2 Choosing the augmenting path

Algorithm	Path selection	Complexity	Note
Ford–Fulkerson	Any augmenting path	$O(E \cdot f^*)$	Depends on capacity <i>values</i> . With irrational capacities it may fail to terminate at all — a good reminder that termination needs proving.
Edmonds–Karp	Shortest (fewest edges), via BFS	$O(VE^2)$	A one-line change from the above, and the bound becomes independent of capacities.
Dinic	Blocking flows on a level graph	$O(V^2E)$	$O(E\sqrt{V})$ on unit-capacity graphs, which makes it the practical choice for bipartite matching.
Push–relabel	Local pushes, no path search	$O(V^3)$	Often fastest in practice on dense graphs.

3 Reductions: the actual point

Flow is rarely the question anyone asks. Its importance is that an implausible range of problems reduce to it, and the reduction is usually the hard and interesting step.

3.1 Bipartite matching

Given sets L and R and a set of permitted pairs, find the largest collection of pairs using no element twice.

- 1 Add a source s with a capacity-1 edge to each vertex of L .
- 2 Keep each permitted pair as a capacity-1 edge from L to R .
- 3 Add a sink t with a capacity-1 edge from each vertex of R .
- 4 Compute the maximum flow. Its value is the size of the maximum matching, and the saturated L – R edges are the matching.

WHY THE FLOW IS AUTOMATICALLY A MATCHING

The **integrality theorem**: if all capacities are integers, there is a maximum flow in which every edge carries an integer amount, and augmenting-path algorithms find such a flow.

With unit capacities, every edge therefore carries 0 or 1, so the solution is a genuine set of disjoint pairs rather than a fractional assignment. This is not an accident of the construction — it is a property of flow that makes a great many combinatorial reductions work.

3.2 Vertex capacities

Flow constrains edges. To constrain a *vertex* — a router handling limited traffic, a worker taking at most three tasks, a path visiting each node once — split it:

$$v \rightarrow v_{\text{in}} \rightarrow v_{\text{out}}, \quad \text{capacity on the connecting edge}$$

Redirect every incoming edge to v_{in} and every outgoing edge from v_{out} . Any path through v must traverse the connecting edge, so its capacity bounds the traffic through the vertex. This single trick brings vertex-disjoint paths, node capacities, and several scheduling constraints into the flow framework.

3.3 Min cut as the answer

Several important applications want the cut rather than the flow.

Application	Construction	What the min cut gives
Image segmentation	Each pixel is a vertex. Edges to the source and sink weighted by how much the pixel looks like foreground or background. Edges between neighbouring pixels weighted by similarity.	The labelling that best balances per-pixel evidence against boundary smoothness. This is the graph-cuts method, standard in computer vision.

Application	Construction	What the min cut gives
Project selection	Profitable projects joined to the source with their profit; costly prerequisites joined to the sink with their cost; infinite-capacity edges encoding dependencies.	The maximum-profit set of projects that is closed under its prerequisites.
Baseball elimination	Remaining games as flow that must be distributed among teams.	Whether a given team can still finish first — a feasibility question, answered by whether the flow saturates.

4 Recognising a flow problem

The algorithms here are in every library. The skill worth developing is noticing that a problem *is* a flow problem, which is frequently well disguised.

- Something is **assigned** or **routed** subject to capacity limits.
- There is a **bipartite** structure: two groups and a set of permitted pairings.
- You are asked to **separate** two things as cheaply as possible — that is a min cut.
- **Conservation** arises naturally: whatever enters a node leaves it.
- A **greedy approach almost works** but gets permanently stuck on an early commitment. This is the strongest signal of all, because flow is exactly greedy with the ability to undo.

THE HONEST SUMMARY

You will almost never implement a max-flow algorithm from scratch — good implementations exist everywhere.

What you will do, if you have learned this module properly, is look at a scheduling, allocation, or separation problem that nobody described as a graph problem and realise it is one. That recognition is worth far more than the ability to code Dinic's algorithm from memory.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Network Flow, Max-Flow Min-Cut	Two lectures: the theorem with proof, then the reductions.
Jeff Erickson, Algorithms — Maximum Flow and Applications chapters	The best free treatment of the reductions, which is the part worth studying hardest.

Resource	What it covers for this module
Boykov & Kolmogorov — graph cuts in vision (free preprint)	How min cut became a standard tool in image segmentation. Directly relevant if you are heading toward CSCE 748.
CSES Problem Set — Graph section (flow problems)	Auto-graded practice on matching, disjoint paths, and min cut.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Ford–Fulkerson with DFS path selection, then Edmonds–Karp with BFS. Construct a graph where the DFS version takes many more augmentations and measure the difference.
- 2 Build the classic example where Ford–Fulkerson with a bad path choice takes a number of augmentations proportional to the capacity value, and confirm Edmonds–Karp avoids it.
- 3 Remove reverse residual edges from your implementation and find a graph where it now returns a suboptimal flow. This is the clearest way to understand why they exist.
- 4 Reduce bipartite matching to flow and solve a job-assignment instance. Verify the resulting flow is integral.
- 5 Implement vertex splitting and solve a vertex-disjoint paths problem.
- 6 Implement a two-label image segmentation by min cut on a small greyscale image. Use intensity difference for neighbour weights, and report how the smoothness weight changes the result.
- 7 Extract the minimum cut from your final residual graph and verify its capacity equals the max flow value.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the max-flow min-cut theorem. Which direction is easy, and why?
- 2 What are residual reverse edges and what would go wrong without them?
- 3 What single change turns Ford–Fulkerson into Edmonds–Karp, and what does it buy?
- 4 Describe the reduction from bipartite matching to flow, and explain why the resulting flow is a genuine matching.
- 5 How do you impose a capacity on a vertex rather than an edge?
- 6 Give three signals that a problem you are looking at might be a flow problem.